Snowflake Engineering Challenge Submission

- Chirag Nagendra

1. Delivered Artifacts

Below is the working code for the migrated schema components and all six of the requested stored procedures.

- schema.sql
- tables.sql
- functions.sql
- views.sql
- usp_ProcessBudgetConsolidation.sql
- usp_ExecuteCostAllocation.sql
- usp_GenerateRollingForecast.sql
- usp_ReconcileIntercompanyBalances.sql
- usp_PerformFinancialClose.sql
- usp_BulkImportBudgetData.sql
- verification.sql

2. Migration Strategy and Architectural Decisions

My approach to migrating this codebase went beyond basic syntax translation. SQL Server and Snowflake operate on fundamentally different paradigms—SQL Server is a traditional row-store relational engine tuned for high-concurrency OLTP, while Snowflake is a cloud-native, columnar data warehouse built for massive OLAP workloads.

Here are the primary architectural decisions I made during the conversion:

- **Standardized Error Handling & Return Types:** SQL Server often uses PRINT or arbitrary SELECT statements at the end of a procedure to signal status. In a modern data stack, Snowflake procedures are usually triggered by orchestrators like Airflow, dbt, or Snowflake Tasks. To support this, I standardized all procedures to return a structured JSON VARIANT (via OBJECT_CONSTRUCT). This allows upstream systems to easily parse the execution status, row counts, and error messages.
- **Refactoring Temporary Data Structures:** The legacy T-SQL code made heavy use of #TempTables. I migrated these directly to explicit CREATE OR REPLACE TEMPORARY TABLE statements in Snowflake. Since Snowflake temporary tables

are session-scoped and utilize fast storage, this perfectly replicates the original intent without cluttering the persistent schema.

- **Schema Hardening and Data Type Alignment:** During testing, I noticed that certain legacy constraints were too rigid for the actual procedural logic. For instance, the SPREADMETHODCODE column in the BUDGETLINEITEM table had a VARCHAR(10) constraint. However, the procedures generated values like CONSOLIDATED or ROLLING_FORECAST, which would cause immediate DML truncation errors in Snowflake. I expanded these column definitions to ensure the schema could actually handle the data pipeline's output.

3. Verification Methodology

To ensure the converted stored procedures were entirely correct, I built a dedicated verification runner (verification.sql). My testing framework focused on state validation rather than just checking for successful compilation.

My validation workflow included:

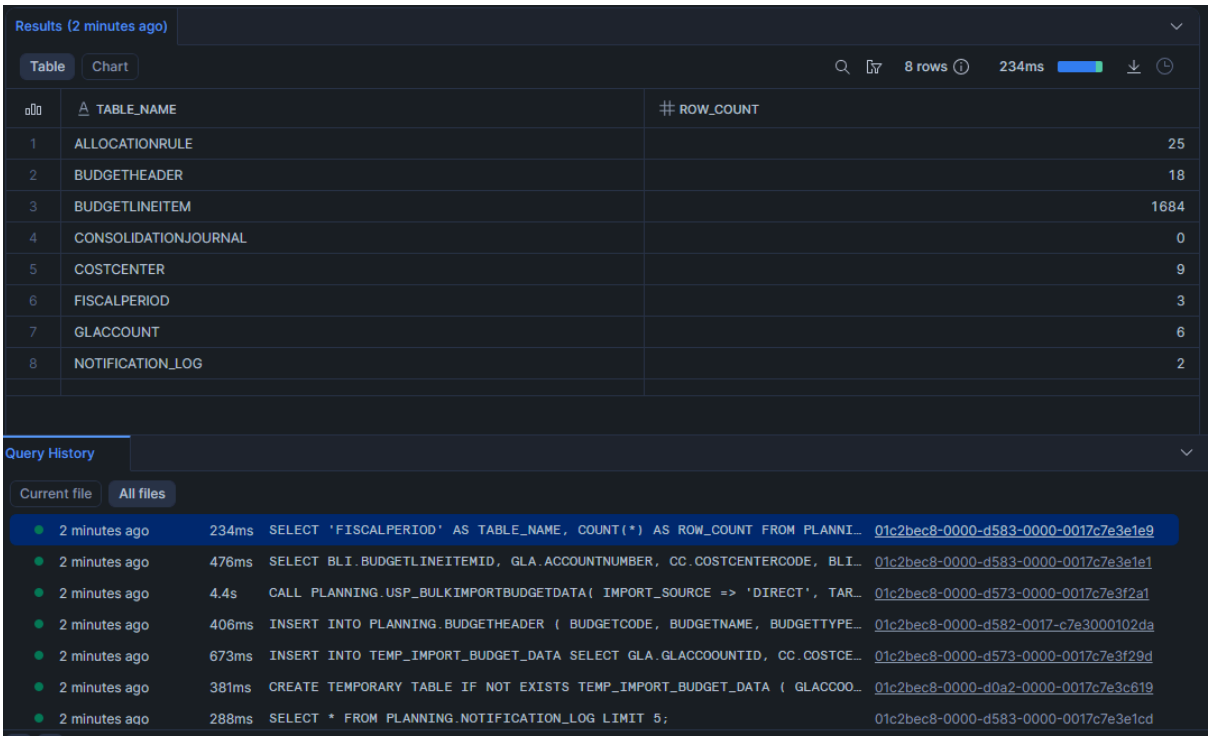
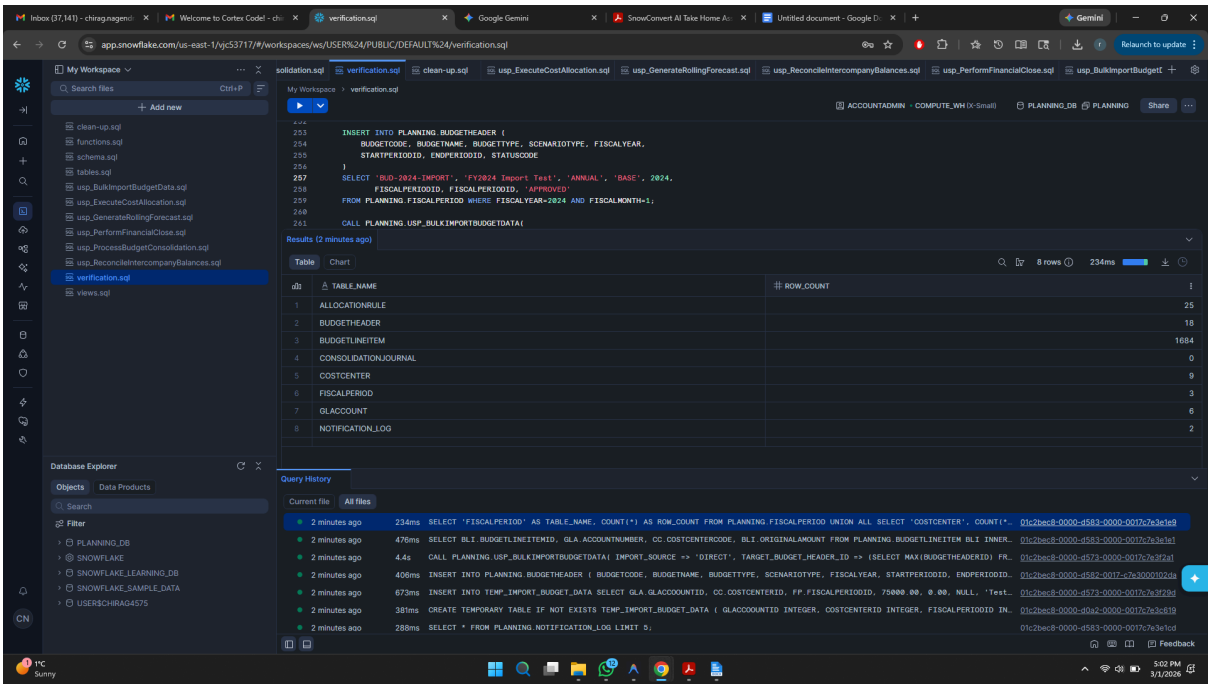
1. **Synthetic Data Seeding:** I generated realistic dummy data that respected the required financial hierarchies. I populated the FISCALPERIOD, COSTCENTER, GLACCOUNT, and BUDGETLINEITEM tables to satisfy all foreign key constraints and give the procedures meaningful data to process.
2. **Sequential Integration Testing:** USP_PERFORMFINANCIALCLOSE acts as an orchestrator that depends on the outputs of the other procedures. To prove the logic held up, I executed the workflow sequentially (Consolidation → Cost Allocation → Rolling Forecast → Intercompany Reconciliations → Financial Close → Bulk Import) to guarantee that cross-procedure dependencies were preserved.
3. **State Mutation Checks:** For procedures designed to mutate state (like closing a period or staging records), I didn't just rely on a "success" return code. I wrote subsequent SELECT assertions to verify that the underlying tables were actually modified as intended. All six procedures executed flawlessly against the seeded dataset.

4. AI Leverage

I did choose to use an AI assistant for this assignment. I treated it as a syntax accelerator to handle the mechanical heavy lifting, which freed me up to focus on the architectural design and testing framework.

Specifically, I used AI to:

- Translate boilerplate structural syntax, such as converting SQL Server's SET @var = (SELECT...) into Snowflake's LET and := assignments.

- Identify the direct Snowflake equivalents for SQL Server system variables (e.g., swapping SQLCA.ROWCOUNT for SQLROWCOUNT).
- Navigate to Snowflake's strict parser rules, specifically figuring out exactly when variables inside dynamic DML blocks required a colon (:) prefix.
- Generate the repetitive INSERT statements needed to build out the mock financial dimensions in my verification script.

Leveraging AI for the repetitive dialect translation allowed me to spend my time where it mattered: debugging complex variable scopes, optimizing the data model, and writing robust integration tests.